Lab 04. Thiết kế bộ tăng tốc Conv1 INT8 bằng Vitis HLS và Vivado cho PYNQ-Z2

Step 1. Xác định phần CNN cần tăng tốc trên FPGA

Sử dụng Tiny CNN V5 đã huấn luyện trong Lab 03 để lựa chọn tầng đầu tiên cho bộ tăng tốc phần cứng: Conv2D 3 × 3, 8 bộ lọc, kích hoạt ReLU và MaxPooling2D 2 × 2, stride = 2. Phần còn lại của mạng dự kiến được thực thi trên bộ xử lý ARM Cortex-A9 ở Lab 05.

Thuộc tính	Giá trị
Ảnh đầu vào	96 × 96 × 1; grayscale, INT8
Conv1	Kernel 3 × 3; 8 filters; padding valid; stride 1
Đầu ra Conv1	94 × 94 × 8
Đầu ra sau ReLU và MaxPool	47 × 47 × 8
Số phần tử kết quả	47 × 47 × 8 = 17.672 giá trị INT8

Dữ liệu vào gồm 96 × 96 = 9.216 byte INT8. Mỗi vị trí đầu ra chứa 8 kênh, được đóng gói thành một từ AXI4-Stream 64-bit; tổng cộng phát 47 × 47 = 2.209 từ (17.672 byte).

Step 2. Chuẩn bị trọng số và tham số lượng tử hóa INT8

Từ mô hình tiny_cnn_v5_int8.tflite, sử dụng script extract_conv1.py để lấy trọng số, bias và tham số requantization của Conv1. Trọng số TFLite có cấu trúc [8, 3, 3, 1]; khi đưa vào HLS, loại bỏ chiều input_channel bằng 1 để tạo mảng conv1_w_int8[8][3][3].

- Input zero-point = -128; input scale xấp xỉ 0,0039215689.
- Output zero-point = -128; output scale xấp xỉ 0,0039095692.
- Bias sử dụng INT32; trọng số INT8 lượng tử hóa theo từng kênh đầu ra.
- Tập tin conv1_int8_params.h chứa mảng weight, bias, multiplier và shift cho 8 kênh.

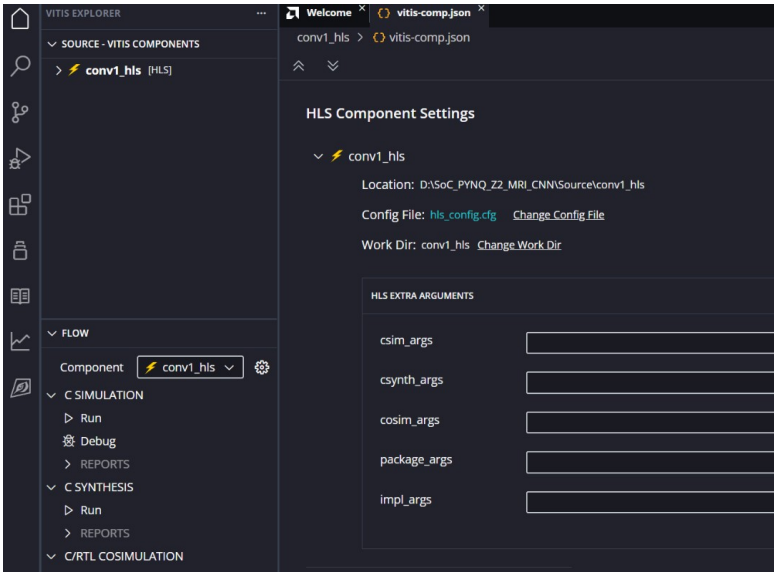
```
#define CONV1_IN_ZERO_POINT  (-128)
#define CONV1_OUT_ZERO_POINT (-128)
#define CONV1_FILTERS 8
#define CONV1_KERNEL  3
static const int8_t conv1_w_int8[8][3][3] = { /* ... */ };
static const int32_t conv1_b_int32[8] = { /* ... */ };
```

Cấu hình lượng tử hóa INT8 trên là cấu hình cuối cùng cho IP. Phiên bản thông số Q1.15 thử nghiệm trước đó không dùng để tạo Bitstream này.

Step 3. Tạo HLS Component trong Vitis Unified IDE 2026.1

Mở Vitis Unified IDE 2026.1 và tạo HLS Component conv1_hls tại D:\SoC_PYNQ_Z2_MRI_CNN\Source\conv1_hls. Chọn part xc7z020clg400-1 tương ứng PYNQ-Z2, clock target 10 ns (100 MHz) và định dạng đóng gói ip_catalog.

Thêm conv1.cpp và conv1_int8_params.h vào Sources; thêm conv1_tb.cpp vào Test Bench. Lựa chọn conv1_hls làm top-level function. Các file được lưu trong Source để phục vụ việc tái tạo IP.



Hình 4.1. Cấu hình HLS Component conv1_hls trong Vitis Unified IDE.

Step 4. Thiết lập giao tiếp cho bộ tăng tốc Conv1

IP sử dụng AXI4-Stream để nhận và xuất dữ liệu, đồng thời có giao tiếp AXI4-Lite để ARM điều khiển tín hiệu ap_start, ap_done và ap_idle. Mỗi chu kỳ truyền vào chứa 1 pixel INT8; mỗi từ đầu ra chứa 8 kênh INT8 đã pooling.

```
typedef ap_axiu<8, 0, 0, 0> in_axis_t;
typedef ap_axiu<64, 0, 0, 0> out_axis_t;

void conv1_hls(hls::stream<in_axis_t>& s_axis,
           hls::stream<out_axis_t>& m_axis) {
#pragma HLS INTERFACE mode=axis port=s_axis
#pragma HLS INTERFACE mode=axis port=m_axis
#pragma HLS INTERFACE mode=s_axilite port=return bundle=control
   // Streaming computation
}
```

INTERFACE	REGISTER	OFFSET	WIDTH	ACCESS	DESCRIPTION
s_axi_control	CTRL	0x00	32	RW	Control signals
s_axi_control	GIER	0x04	32	RW	Global Interrupt Enable Register
s_axi_control	IP_IER	0x08	32	RW	IP Interrupt Enable Register
s_axi_control	IP_ISR	0x0c	32	RW	IP Interrupt Status Register

AXIS

INTERFACE	DIRECTION	REGISTER MODE	TDATA	TKEEP	TLAST	TREADY
m_axis	out	both	64	8	1	1
s_axis	in	both	8	1	1	1

TOP LEVEL CONTROL

Hình 4.2. Báo cáo giao tiếp AXI-Lite, AXI4-Stream và các cổng điều khiển.

Step 5. Cài đặt Conv1, ReLU và requantization

Thiết kế lưu hai hàng ảnh trước đó bằng line buffer; cửa sổ trượt 3 × 3 được cập nhật ngay khi nhận pixel. Với 8 bộ lọc, các vòng lặp tính tích chập được unroll để tăng mức độ song song. Kết quả tích lũy INT32 được requantize theo multiplier/shift từng kênh và cộng zero-point đầu ra, sau đó giới hạn trong [-128, 127].

```
#pragma HLS PIPELINE II=1
// At each valid 3x3 window:
ap_int<32> acc = conv1_b_int32[f];
acc += (ap_int<32>)prod;
ap_int<32> q = requantize(
   acc, conv1_multiplier[f], conv1_shift[f]);
q += CONV1_OUT_ZERO_POINT;
if (q < -128) q = -128;
if (q > 127) q = 127;
```

Ở cấu hình hiện tại, mức zero-point -128 cũng chính là biểu diễn của giá trị thực bằng 0. Việc clamp về INT8 đồng thời thực hiện kích hoạt ReLU đã được hợp nhất vào đầu ra Conv1.

Step 6. Cài đặt MaxPooling 2 × 2 và tín hiệu TLAST

Đối với mỗi ô pooling, lưu hàng Conv1 phía trên và giá trị phía dưới bên trái; khi nhận giá trị dưới bên phải, chọn lớn nhất theo từng kênh. Không cần lưu toàn bộ tensor 94 × 94 × 8 trong BRAM. Tín hiệu TLAST chỉ được kích hoạt ở từ đầu ra cuối cùng của ảnh.

Kết quả là 2.209 từ 64-bit cho mỗi ảnh, đúng 17.672 byte dữ liệu. Đóng gói theo byte cần được giải mã thống nhất ở Python khi triển khai trên PYNQ.

Step 7. Kiểm tra đúng chức năng bằng C Simulation

Chạy C Simulation với conv1_tb.cpp. Testbench tạo ảnh INT8 96 × 96 có giá trị xác định, tái tính Conv1, ReLU và MaxPool bằng thuật toán CPU độc lập, sau đó so sánh toàn bộ dữ liệu theo từng kênh và kiểm tra TLAST.

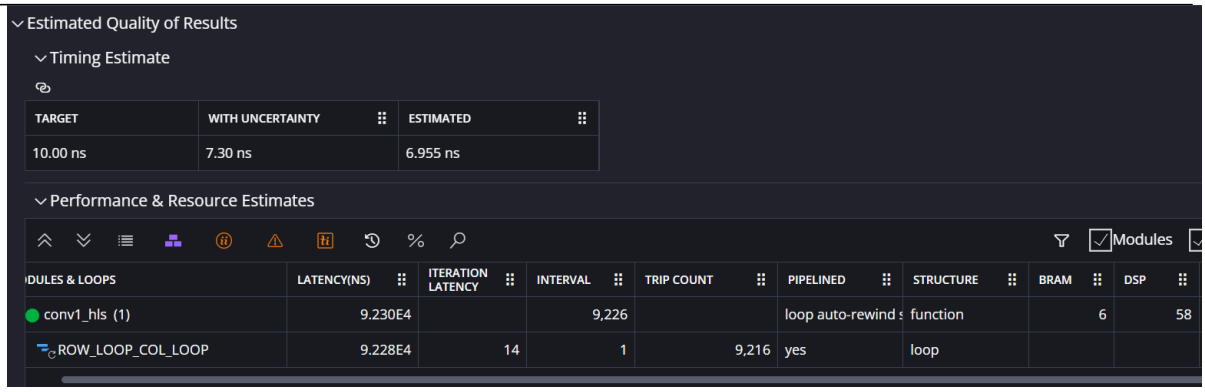
```
Verified 2209 pooled vectors, 17672 INT8 channels; errors=0
PASS: Conv1 + ReLU + MaxPool INT8 CPU vs HLS C simulation
CSim done with 0 errors
```

Kết quả xác nhận HLS C và chương trình CPU reference trong testbench nhất quán. Chưa thể từ đó kết luận đầu ra đã khớp từng bit với TFLite Interpreter thực tế; phép đối chiếu này sẽ tiếp tục thực hiện ở Lab 05.

Step 8. Chạy C Synthesis và đánh giá HLS

Chạy C Synthesis với clock target 10 ns. Vitis HLS ước lượng period 6,955 ns, II = 1 cho vòng lặp đọc 9.216 pixel, và độ trễ toàn chức năng khoảng 9.230 chu kỳ. Tại clock 100 MHz, độ trễ tính toán ước lượng xấp xỉ 92,3 µs cho một ảnh; con số này không bao gồm chi phí DMA, điều khiển hay phần mềm.

Tham số HLS	Giá trị báo cáo
Clock target / Estimated period	10 ns / 6,955 ns
Vòng lặp xử lý ảnh	9.216 iterations; II = 1
Latency ước lượng	~9.230 chu kỳ
Tài nguyên HLS ước lượng	LUT 5.493; FF 4.856; BRAM 6; DSP 58



Hình 4.3. C Synthesis: ước lượng thời gian, II và tài nguyên IP Conv1.

Step 9. Chạy C/RTL Co-Simulation và đóng gói IP

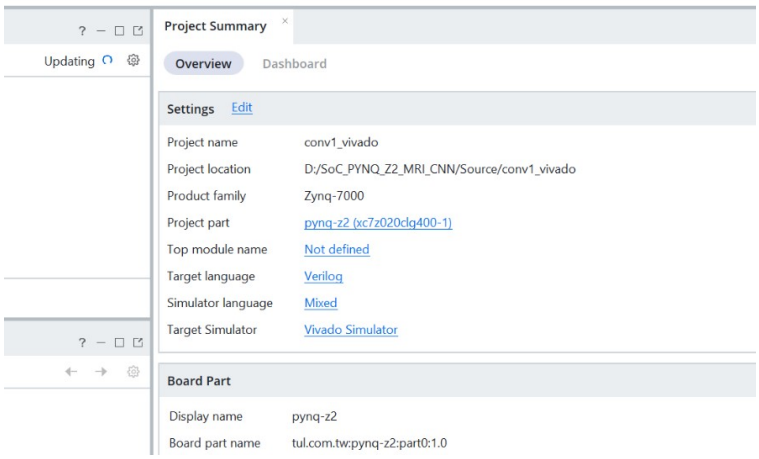
Sau khi C Synthesis thành công, chạy C/RTL Co-Simulation để kiểm tra hoạt động giữa mô hình HLS và RTL được sinh ra. Báo cáo Co-Simulation ghi nhận

latency khoảng 9.228 chu kỳ. Đây là kết quả mô phỏng RTL, không phải thời gian đo trên board.

Thực hiện Package IP ở chế độ ip_catalog. Vitis HLS thông báo Package finished successfully và tạo component.xml cùng gói xilinx_com_hls_conv1_hls_1_0.zip tại Source\conv1_hls\conv1_hls\hls\impl\ip.

Step 10. Tạo dự án Vivado và đăng ký IP Repository

Mở Vivado 2026.1, tạo project conv1_vivado trong thư mục Source\conv1_vivado. Chọn Board Part tul.com.tw:pynq-z2:part0:1.0, tương ứng FPGA xc7z020clg400-1. Thêm đường dẫn thư mục HLS impl\ip vào Project Settings → IP → Repository, xác nhận IP xilinx.com:hls:conv1_hls:1.0 xuất hiện trong IP Catalog.

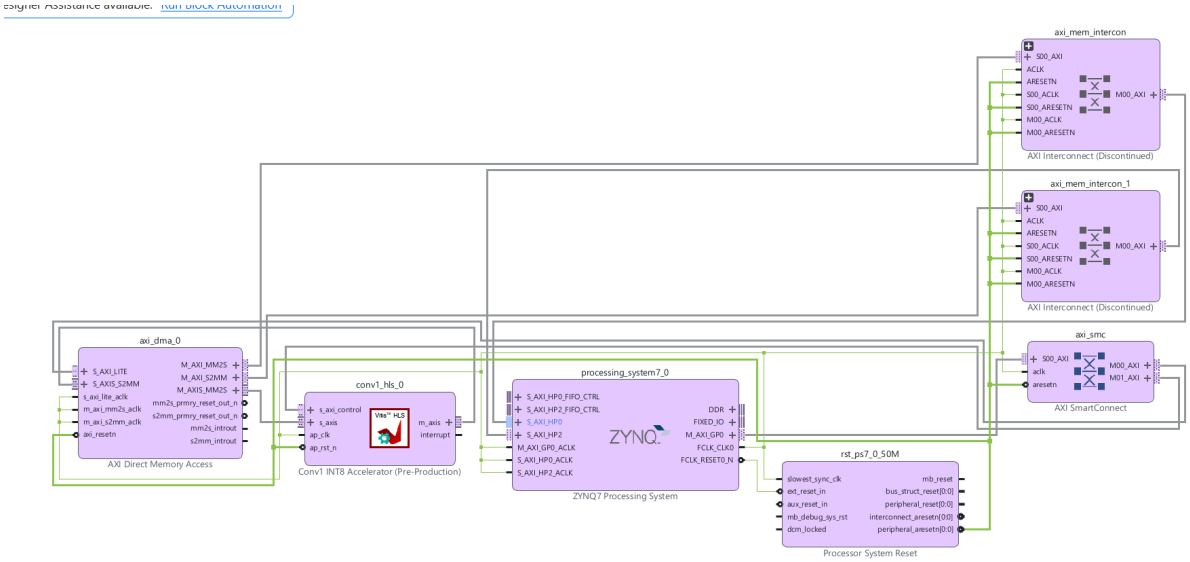


Hình 4.4. Vivado nhận đúng Board Part pynq-z2 và chip Zynq-7020.

Step 11. Tạo Block Design và kết nối luồng dữ liệu AXI

Tạo Block Design conv1_system. Thêm processing_system7_0, axi_dma_0 và conv1_hls_0. Kết nối AXI DMA M_AXIS_MM2S đến conv1_hls_0/s_axis; kết nối conv1_hls_0/m_axis đến AXI DMA S_AXIS_S2MM.

Đường điều khiển DMA/S_AXI_LITE và Conv1/s_axi_control được đưa tới PS/M_AXI_GP0 thông qua AXI SmartConnect. Đường MM2S đọc RAM đi qua PS/S_AXI_HP0; đường S2MM ghi RAM đi qua PS/S_AXI_HP2. Nhờ vậy ARM có thể khởi chạy IP và truy cập bộ đệm trong bộ nhớ hệ thống.

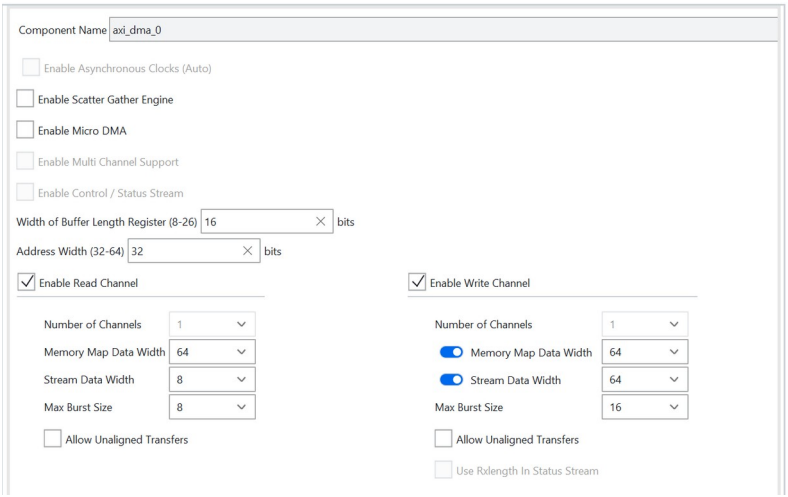


Hình 4.5. Block Design Zynq PS, AXI DMA, Conv1 HLS, AXI Interconnect và Reset.

Step 12. Cấu hình AXI DMA theo kích thước luồng INT8

Tại IP axi_dma_0, tắt Scatter Gather để dùng Simple DMA. Bật cả Read Channel và Write Channel; address width 32-bit, memory-map data width 64-bit. Chọn stream MM2S width 8-bit và S2MM width 64-bit, tương ứng với cổng vào/ra của IP Conv1.

Thiết lập Width of Buffer Length Register = 16 bit. Dữ liệu đầu ra 17.672 byte vượt mức tối đa 16.383 byte của thanh ghi độ dài 14-bit; do đó phải dùng ít nhất 15-bit, và lựa chọn 16-bit được sử dụng trong thực hành.



Hình 4.6. Cấu hình Simple DMA: MM2S 8-bit, S2MM 64-bit và Buffer Length 16-bit.

Step 13. Cấu hình PS chuẩn PYNQ-Z2 và xử lý lỗi DDR ban đầu

Kiểm tra PS bằng lệnh get_property CONFIG.PCW_UIPARAM_DDR_BUS_WIDTH cho thấy giá trị ban đầu là 32 Bit,

không phù hợp với board PYNQ-Z2. Trước khi sửa, sử dụng write_bd_tcl để sao lưu thiết kế conv1_system_before_preset.tcl.

Xác định chính xác Board Part thông qua get_board_parts và thuộc tính FILE_NAME để tìm preset.xml đi kèm Vivado Board Store. File preset xác nhận DDR Bus Width = 16 Bit, Part Number = MT41J256M16 RE-125 và PS Input Clock = 50 MHz. Áp dụng bộ tham số CONFIG.PCW_* trong preset, đồng thời giữ bật GP0, HP0, HP2 và cấu hình FCLK_CLK0 100 MHz.

Thông số PS sau cập nhật	**Giá trị đọc lại từ Vivado**
PCW_UIPARAM_DDR_BUS_WIDTH	16 Bit
PCW_UIPARAM_DDR_PARTNO	MT41J256M16 RE-125
PCW_CRYSTAL_PERIPHERAL_FREQMHZ	50
PCW_USE_M_AXI_GP0 / HP0 / HP2	1 / 1 / 1
PCW_FPGA0_PERIPHERAL_FREQMHZ	100

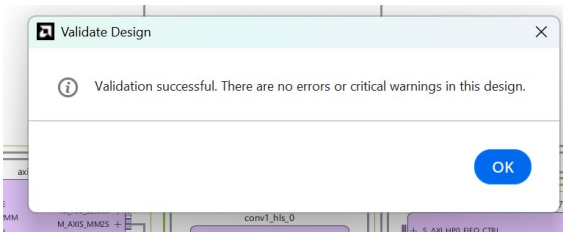
Việc sử dụng preset đầy đủ nhằm duy trì các thông số DDR và MIO đồng bộ với board, thay vì chỉ đổi riêng độ rộng bus.

Step 14. Kiểm tra Clock, Reset và Validate Design

Kết nối FCLK_CLK0 của PS ở 100 MHz tới m_axi_mm2s_aclk, m_axi_s2mm_aclk và s_axi_lite_aclk của DMA; ap_clk của Conv1; slowest_sync_clk của Processor System Reset; và M_AXI_GP0_ACLK, S_AXI_HP0_ACLK, S_AXI_HP2_ACLK của PS.

Kết nối PS/FCLK_RESET0_N đến ext_reset_in của reset controller; cấp peripheral_aresetn cho axi_dma_0/axi_resetn và conv1_hls_0/ap_rst_n. Tcl Console xác nhận các chân được kiểm tra có cùng tên net clock/reset tương ứng.

Chạy validate_bd_design. Vivado ghi nhận thiết kế hợp lệ; không có error hay critical warning trong lần Validate đã báo cáo. Việc này chỉ xác nhận cấu trúc sơ đồ, chưa phải kiểm thử chạy trên PYNQ-Z2.



Hình 4.7. Kết quả Validate Design thành công trong Vivado.

Step 15. Tạo HDL Wrapper và Generate Output Products

Lưu Block Design bằng save_bd_design. Trong Sources, nhấp phải conv1_system.bd → Create HDL Wrapper; chọn Let Vivado manage wrapper and auto-update. Vivado tạo top-level conv1_system_wrapper dùng để tổng hợp và triển khai thiết kế.

Tiếp theo chọn Generate Output Products ở conv1_system.bd để xây dựng HDL và các file IP cần thiết. Bước này hoàn tất trước khi chạy toàn bộ flow Synthesis và Implementation.

Step 16. Chạy Vivado Synthesis và đọc bảng Utilization

Chạy Run Synthesis và mở Synthesized Design. Trong báo cáo Utilization, thiết kế tổng thể sử dụng 4.405 LUT, 4.484 FF, 6 BRAM tile, 62 DSP và 1 BUFG. Các giá trị sử dụng đều thấp hơn tổng tài nguyên logic của Zynq-7020.

Tài nguyên	Đã dùng	Tổng có	Tỷ lệ
Slice LUT	4.405	53.200	8,28%
Slice Register	4.484	106.400	4,21%
Block RAM Tile	6	140	4,29%
DSP	62	220	28,18%
BUFGCTRL	1	32	3,13%

Riêng conv1_hls_0 chiếm 2.070 LUT, 1.567 FF, 3 BRAM tile và 62 DSP. Bảng Synthesis cũng báo 130 Bonded IOPADs; chỉ số này liên quan đến cách công cụ hạch toán chân I/O của thiết kế Zynq, không thể tự suy ra PL đã hết chân sử dụng.

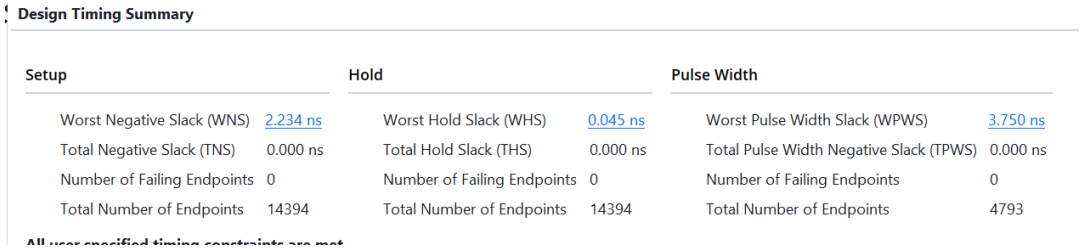
Name	Slice LUTs (53200)	Slice Registers (106400)	F7 Muxes (26600)	Block RAM Tile (140)	DSPs (220)	Bonded IOPADs (130)	BUFGCTRL (32)
conv1_system_wrapper	4405	4484	13	6	62	130	1
conv1_system_i (conv1_system)	4405	4484	13	6	62	0	1
axi_dma_0 (conv1_system_axi_	1650	2155	13	3	0	0	0
axi_mem_intercon (conv1_syste	46	85	0	0	0	0	0
axi_mem_intercon_1 (conv1_sy	196	203	0	0	0	0	0
axi_smc (conv1_system_axi_sm	404	441	0	0	0	0	0
conv1_hls_0 (conv1_system_co	2070	1567	0	3	62	0	0

Hình 4.8. Bảng Utilization của thiết kế conv1_system_wrapper sau Synthesis.

Step 17. Đánh giá Timing sau Synthesis

Mở Report Timing Summary. Ở giai đoạn Synthesis, WNS = +2,234 ns; TNS = 0,000 ns; WHS = +0,045 ns; THS = 0,000 ns; WPWS = +3,750 ns và không có failing

endpoint trong báo cáo. Điều này cho thấy thiết kế đạt các ràng buộc Timing tại thời điểm tổng hợp.



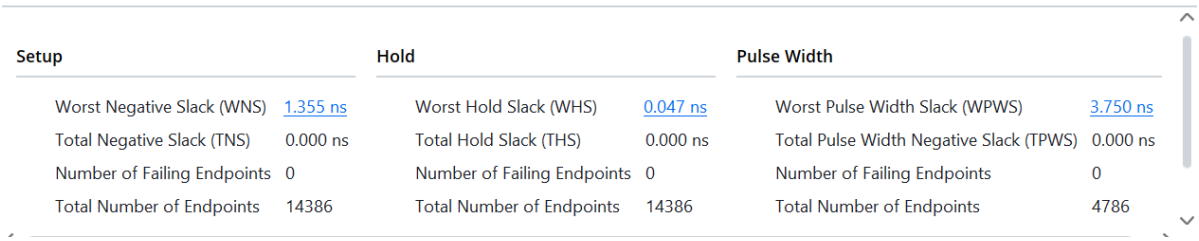
Hình 4.9. Design Timing Summary sau Synthesis.

Step 18. Run Implementation và kiểm tra Timing sau đi dây

Chạy Run Implementation để Vivado đặt và nối dây các tài nguyên logic trên chip. Sau khi hoàn thành, mở Implemented Design và chạy report_timing_summary. Các chỉ số cuối Implementation như sau:

Chỉ số	Sau Synthesis	Sau Implementation
WNS (Setup)	+2,234 ns	+1,355 ns
TNS	0,000 ns	0,000 ns
WHS (Hold)	+0,045 ns	+0,047 ns
THS	0,000 ns	0,000 ns
WPWS	+3,750 ns	+3,750 ns
Failing endpoints	0	0

WNS sau Implementation vẫn dương 1,355 ns, không có failing endpoint. Vì kết quả này đã phản ánh Placement & Routing, có thể kết luận thiết kế đáp ứng các ràng buộc Timing được khai báo cho mục tiêu FCLK 100 MHz. Kết luận không đồng nghĩa đã đo xung hoặc thông lượng thực trên board.



Hình 4.10. Design Timing Summary sau Implementation với WNS dương.

Step 19. Kiểm tra DRC và Generate Bitstream

Chạy report_drc trong Implemented Design. Tcl Console ghi nhận report_drc completed successfully và giá trị trả về 0. Cần phân biệt giá trị trả về Tcl với tổng số

vi phạm DRC; tài liệu thực hành này không khẳng định số vi phạm là 0 khi chưa đọc bảng DRC violations riêng.

Tiến hành Generate Bitstream. Theo dõi run impl_1 tới bước write_bitstream. Sau khi thực hiện, kiểm tra trạng thái bằng Tcl:

```
puts "STATUS: [get_property STATUS [get_runs impl_1]]"
puts "PROGRESS: [get_property PROGRESS [get_runs impl_1]]"
# Kết quả ghi nhận:
STATUS: write_bitstream Complete!
PROGRESS: 100%
```

Như vậy file Bitstream đã được tạo thành công. Kết quả chưa chứng minh thuật toán hoạt động đúng trên FPGA thật; việc nạp và kiểm tra chức năng sẽ diễn ra trong Lab 05.

Step 20. Lấy file Bitstream và Hardware Handoff

Sau khi Generate Bitstream, tìm hai file tại thư mục project Vivado. Bản gốc Bitstream có tên conv1_system_wrapper.bit; Hardware Handoff có tên conv1_system.hwh. Để PYNQ Overlay tìm đúng cặp file, sao chép và đặt tên gốc thống nhất là conv1_accel.

```
set src "D:/SoC_PYNQ_Z2_MRI_CNN/Source/conv1_vivado"
set dst "D:/SoC_PYNQ_Z2_MRI_CNN/Source"
file copy -force  "$src/conv1_vivado.runs/impl_1/conv1_system_wrapper.bit"  "$dst/conv1_accel.bit"
file copy -force  "$src/conv1_vivado.gen/sources_1/bd/conv1_system/hw_handoff/conv1_system.hwh"
"$dst/conv1_accel.hwh"
```

File	Kích thước xác nhận	Thư mục đích
conv1_accel.bit	4.045.775 byte	Source
conv1_accel.hwh	331.944 byte	Source

Các tệp gốc vẫn nằm trong project Vivado; thao tác trên chỉ sao chép. Trên PC kết nối PYNQ-Z2, hai file .bit và .hwh phải đặt cùng thư mục và giữ cùng tên gốc conv1_accel.

Step 21. Ghi nhận kết quả Lab 04 và giới hạn kiểm chứng

Lab 04 đã hoàn thành phần tạo IP Conv1 INT8 bằng Vitis HLS, kiểm chứng C/RTL theo phạm vi thực hành, tích hợp PS - AXI DMA - IP, tổng hợp và triển khai Vivado đạt Timing, rồi xuất cặp file conv1_accel.bit và conv1_accel.hwh.

- Đã kiểm chứng: C Simulation khớp CPU reference 2.209 vector và 17.672 giá trị INT8, errors = 0.

- Đã ước lượng: HLS khoảng 9.230 chu kỳ, tương đương 92,3 µs tại 100 MHz (không phải số đo thực tế).
- Đã xác nhận: Implementation WNS +1,355 ns; Bitstream write_bitstream Complete!, progress 100%.
- Chưa kiểm chứng: sai số từng phần tử so với TensorFlow Lite Runtime trên ảnh thật, truyền DMA và tốc độ đo trên PYNQ-Z2, ghép ARM với các lớp CNN còn lại.

Lab 05 sẽ nạp Overlay từ cặp file vừa tạo, thử nghiệm truyền dữ liệu qua AXI DMA, đối chiếu đầu ra Conv1 với phần mềm và hoàn thiện luồng suy luận CNN kết hợp ARM + FPGA. Chỉ sau khi có kết quả trên phần cứng mới đánh giá được mức tăng tốc thực tế.